PI_Curs_04

4.0

Bun venit la o nouă etapă a studiului nostru despre rețele neuronale! Dacă până acum ne-am concentrat pe date tabulare și date cu structură spațială (imagini) folosind MLP-uri și CNN-uri, lecția 4 introduce o clasă de modele concepute pentru tipul de date care domină lumea reală: **datele secvențiale**.

Ghid de Studiu Complet: 4. Rețele Neuronale Recurente (Recurrent Neural Networks - RNNs)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 4 face tranziția de la rețelele neuronale care presupun că datele de intrare sunt **independente și identic distribuite (i.i.d.)** la **Rețelele Neuronale Recurente (RNNs)**, specializate în gestionarea informațiilor care apar în **ordine secvențială**. Problema centrală a datelor secvențiale (precum cuvintele dintr-o propoziție, cadrele dintr-un video sau datele dintr-o serie de timp) este că înțelegerea lor depinde critic de ordinea apariției. RNN-urile rezolvă această problemă prin introducerea **variabilelor de stare (*state variables*)**. Aceste variabile de stare stochează informațiile din trecut și le combină cu intrările curente pentru a determina ieșirile curente. Pe scurt, în timp ce CNN-urile procesează eficient informația spațială, **RNN-urile sunt concepute pentru a gestiona informația secvențială**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Date Secvențiale:** Majoritatea datelor, spre deosebire de presupunerea i.i.d. a modelelor anterioare, urmează o **ordine secvențială** (ex: cuvinte, audio, serii de timp).
2. **Variabile de Stare:** RNN-urile utilizează **variabile de stare** (H_{t-1}) pentru a stoca informațiile din pașii de timp anteriori.
3. **Calculul Stării Ascunse:** Starea ascunsă actuală (H_t) este o funcție neliniară (ReLU, ϕ) care depinde atât de intrarea curentă (X_t), cât și de starea ascunsă anterioară (H_{t-1}).
4. **Partajarea Ponderilor:** Ponderile (W_{xh}, W_{hh}, W_{hq}) sunt **partajate pe parcursul timpului** (time-step sharing), permițând modelului să învețe tipare care se aplică în întreaga secvență.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4. Rețele Neuronale Recurente

Contrastul cu Modelele Anterioare

- Până acum, au fost întâlnite două tipuri de date: **date tabulare** și **date de tip imagine**.
- Pentru datele de tip imagine, au fost concepute straturi specializate (CNNs) care exploatează **regularitatea** datelor. Dacă s-ar permuta pixelii dintr-o imagine, ar fi mult mai dificil de înțeles conținutul acesteia.
- Cea mai importantă presupunere era că toate exemplele erau **independent și identic distribuite (i.i.d.)**.

Necesitatea RNN-urilor

- Această presupunere i.i.d. **nu este valabilă pentru majoritatea datelor**.
- Exemple de date secvențiale (care urmează o ordine obligatorie): cuvintele dintr-o propoziție, cadrele dintr-un videoclip, semnalul audio dintr-o conversație sau comportamentul de navigare pe un site web.
- Este rezonabil să presupunem că modelele specializate (RNNs) vor descrie mai bine aceste date secvențiale.
- O altă problemă este necesitatea de a **continua o secvență** (ex: continuarea seriei 2, 4, 6, 8, 10...). Acest lucru este frecvent în analiza seriilor de timp, previzionarea pieței bursiere sau accelerarea unui vehicul de curse.
- RNN-urile sunt concepute pentru a gestiona mai bine **informațiile secvențiale**.

Mecanismul de Bază al RNN-urilor

- RNN-urile introduc **variabile de stare (*state variables*)** pentru a stoca informațiile din trecut.
- Aceste variabile de stare sunt combinate cu intrările curente pentru a determina ieșirile curente.

Formulele de Propagare Înainte (Forward Pass)

Pentru un pas de timp t, cu o funcție de activare ϕ care este specificată ca **ReLU** ($\phi(x) = \mathrm{ReLU}(x)$):

1. **Calculul Stării Ascunse (H_t):**

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

2. **Calculul ieșirii (O_t):**

$$O_t = H_t W_{hq} + b_q$$

Dimensionalitatea Tenzorilor (Dacă n este mărimea mini-batch-ului):

Tenzor	Descriere	Dimensiune
X_t	Intrările la momentul t	$n \times d$

Tenzor	Descriere	Dimensiune
W_{xh}	Ponderile de la Intrare la Stare Ascunsă	$d \times h$
W_{hh}	Ponderile de la Stare Ascunsă la Stare Ascunsă	$h \times h$
b_h	Bias-ul Stării Ascunse	$1 \times h$
H_t	Starea Ascunsă la momentul t	$n \times h$
W_{hq}	Ponderile de la Stare Ascunsă la Ieșire	$h \times q$
b_q	Bias-ul Ieșirii	$1 \times q$
O_t	Ieșirea la momentul t	$n \times q$

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Starea Ascunsă Recurentă (H_{t-1})

- **Analogie (Memoria Pe Scurtă Durată):** Starea ascunsă H_{t-1} este ca și cum ai avea o notiță lipită de monitor. Când procesezi informația la momentul t, nu te uiți doar la informația nouă (X_t), ci arunci o privire și pe notiță (H_{t-1}) pentru a vedea ce s-a întâmplat imediat înainte. Apoi, rescrii notița pentru pasul următor (H_t).
- *Asta se întâmplă deoarece:* Acest mecanism permite modelului să dezvolte o "memorie" locală, necesară pentru a înțelege dependențele scurte din cadrul unei secvențe.

Concept Dificil: Non-i.i.d. Data (Date Nesecvențiale vs. Secvențiale)

- **Explicație Simplificată (Diferența cheie):**
  - **I.i.d. (Ex: Imagine):** Dacă aranjați aleatoriu pixelii, imaginea devine de neînțeles, dar *clasa* (e.g., "pisică") nu depinde de ordinea pixelilor, ci de colecția lor.
  - **Secvențial (Ex: Propoziție):** Dacă permutați cuvintele în "Ana are mere," propoziția își pierde sensul. Sensul depinde **în totalitate** de ordinea secvențială. RNN-urile sunt concepute pentru a capta această dependență.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Memorie și Predicție):**
  - **CAUZĂ:** Necesitatea de a prezice elementul următor dintr-o serie de timp (ex: continuarea unei serii 2, 4, 6, 8...).
  - **EFECT:** RNN-urile stochează informațiile din trecut în **variabile de stare** (H_{t-1}), făcând predicția de tip "continuați secvența" realizabilă.

- **Conexiunea cu CNN-urile:** Ambele arhitecturi (CNN și RNN) utilizează **partajarea ponderilor** pentru a eficientiza modelul. În CNN-uri, ponderile sunt partajate spațial (același kernel aplicat peste tot). În RNN-uri, ponderile (W_{xh}, W_{hh}, W_{hq}) sunt partajate **temporal** (aceleași matrice de ponderi sunt folosite la fiecare pas de timp t).

---

5. EXERCITII (Calcul pe Rețele Neuronale Recurente)

Voi genera un exercițiu de calcul complet, bazat pe structura și formulele furnizate în surse.

Exercițiul R.4: Calculul Propagării Înainte (RNN Simplu)

Definiți o Rețea Neuronală Recurentă (RNN) pentru regresie, având **2 intrări** ($d = 2$), o **stare ascunsă de dimensiunea 2** ($h = 2$), și **1 ieșire** ($q = 1$). Funcția de activare pentru starea ascunsă este **ReLU** ($\phi(x) = \max(0, x)$).

Calculați ieșirile O_1 și O_2 pentru doi pași de timp, folosind următoarele intrări și ponderi:

- **Intrări (Batch size $n = 1$):**

$$X_1 = [1 \quad 1], \quad X_2 = [0 \quad 2]$$

- **Starea inițială:** $H_0 = [0 \quad 0]$ (Matrice 1×2)
- **Ponderi generate aleatoriu (din $-1, 0, 1$):**

$$W_{xh} = [1 \quad 0\ 0 \quad 1] \in \mathbb{R}^{2\times 2}, \quad W_{hh} = [1 \quad 0\ 0 \quad -1] \in \mathbb{R}^{2\times 2}$$

$$W_{hq} = [1\ 1] \in \mathbb{R}^{2\times 1}$$

- **Bias-uri:**

$$b_h = [0 \quad 1] \in \mathbb{R}^{1\times 2}, \quad b_q = [-1] \in \mathbb{R}^{1\times 1}$$

1. Calculați H_1 și O_1.
2. Calculați H_2 și O_2.
3. Specificați **numărul total de parametri reali** (greutăți și bias-uri) pentru strat.

Rezolvare R.4:

3. Calculul Total al Parametrilor

- Ponderi W_{xh}: $d \times h = 2 \times 2 = 4$
- Ponderi W_{hh}: $h \times h = 2 \times 2 = 4$
- Bias b_h: $h = 2$
- Ponderi W_{hq}: $h \times q = 2 \times 1 = 2$
- Bias b_q: $q = 1$
- **Total parametri:** $4 + 4 + 2 + 2 + 1 = \mathbf{13}$

1. Pasul de Timp $t = 1$

Starea Ascunsă H_1:

$$H_1 = \phi(X_1 W_{xh} + H_0 W_{hh} + b_h)$$

$$H_1 = \phi\left([1 \quad 1]\,[1 \quad 0\;0 \quad 1] + [0 \quad 0]\,[1 \quad 0\;0 \quad -1] + [0 \quad 1]\right)$$

$$H_1 = \phi\left([1 \quad 1] + [0 \quad 0] + [0 \quad 1]\right)$$

$$H_1 = \phi\left([1 \quad 2]\right)$$

$$H_1 = [\max(0,1) \quad \max(0,2)] = [1 \quad 2]$$

Ieșirea O_1:

$$O_1 = H_1 W_{hq} + b_q$$

$$O_1 = [1 \quad 2]\,[1\;1] + [-1]$$

$$O_1 = [(1 \cdot 1) + (2 \cdot 1)] + [-1]$$

$$O_1 = [3] + [-1] = [2]$$

2. Pasul de Timp $t = 2$

Starea Ascunsă H_2:

$$H_2 = \phi(X_2 W_{xh} + H_1 W_{hh} + b_h)$$

$$H_2 = \phi\left([0 \quad 2]\,[1 \quad 0\;0 \quad 1] + [1 \quad 2]\,[1 \quad 0\;0 \quad -1] + [0 \quad 1]\right)$$

$$H_2 = \phi\left([0 \quad 2] + [(1 \cdot 1 + 2 \cdot 0) \quad (1 \cdot 0 + 2 \cdot (-1))] + [0 \quad 1]\right)$$

$$H_2 = \phi\left([0 \quad 2] + [1 \quad -2] + [0 \quad 1]\right)$$

$$H_2 = \phi\left([1 \quad 1]\right)$$

$$H_2 = [1 \quad 1]$$

Ieșirea O_2:

$$O_2 = H_2 W_{hq} + b_q$$

$$O_2 = [1 \quad 1]\,[1\;1] + [-1]$$

$$O_2 = [(1 \cdot 1) + (1 \cdot 1)] + [-1]$$

$$O_2 = [2] + [-1] = [1]$$

4.1

Suntem acum la un capitol crucial: **4.1 Modele Secvențiale (Sequence Models)**. Această lecție stabilește fundamentele teoretice și practice pentru abordarea datelor care depind de ordinea lor, cum ar fi seriile de timp și textele.

Ghid de Studiu Complet: 4.1 Modele Secvențiale

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 4.1 mută accentul de la datele independente și identic distribuite (i.i.d.) la datele care prezintă o **structură secvențială**. Nevoia de a modela astfel de date, cum ar fi prețul acțiunilor (exemplificat prin indicele **FTSE 100** pe parcursul a aproximativ 30 de ani), impune utilizarea de **instrumente statistice** și noi **arhitecturi de rețele neuronale profunde**. Problema principală în modelarea secvențelor este că numărul de variabile de intrare necesare pentru a prezice o valoare viitoare (x_t) crește cu timpul (t), făcând calculul dificil. Soluția se rezumă la două strategii cheie: simplificarea dependenței istorice (prin modele **autoregresive**) sau utilizarea unei variabile de stare latentă (*summary*) pentru a rezuma trecutul (prin modele **autoregresive latente**).

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Contextul Datelor Secvențiale:** Modelele secvențiale sunt necesare atunci când datele (ex: prețul acțiunilor x_t) depind de ordinea lor în timp.
2. **Problema Dimensională:** Pentru a prezice x_t, este necesară cunoașterea întregii istorii $x_{t-1}, x_{t-2}, \ldots, x_1$. Numărul de intrări variază și crește odată cu timpul t, ceea ce face modelarea computațional nerezolvabilă.
3. **Strategia 1: Modele Autoregresive:** Se presupune că nu este necesară întreaga istorie; este suficientă utilizarea unui interval de timp τ (un *timespan*), bazându-se doar pe $x_{t-1}, \ldots, x_{t-\tau}$.
4. **Strategia 2: Modele Autoregresive Latente:** Se menține un **rezumat ascuns** (h_t) al observațiilor trecute. Predicția $\hat{x}_t$ depinde de acest rezumat latent.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4.1 Modele Secvențiale (Sequence Models)

Definirea și Predictabilitatea Secvențelor

- Se folosesc instrumente statistice și noi arhitecturi neuronale profunde pentru a aborda datele secvențiale.
- Un exemplu folosit pentru simplificare este **prețul acțiunilor (indicele FTSE 100)** observat pe o perioadă de aproximativ 30 de ani.
- Prețul la un moment dat t este notat x_t, unde t este tipic **discret**.
- Un trader care dorește să prezică x_t încearcă să estimeze probabilitatea $P(x_t|x_{t-1}, \ldots, x_1)$, similar cu un model de regresie.
- **Problema majoră:** Numărul de intrări $x_{t-1}, \ldots, x_1$ depinde de t și crește odată cu cantitatea de date, impunând o aproximare pentru a face modelul **computațional tratabil**.

Strategiile de Aproximare

Estimarea eficientă a $P(x_t|x_{t-1}, \ldots, x_1)$ se reduce la două strategii:

1. **Modele Autoregresive (Autoregressive Models):**
   - Se presupune că secvența lungă $x_{t-1}, \ldots, x_1$ nu este necesară în totalitate.
   - Este suficientă utilizarea unui interval de timp de lungime τ, bazându-se doar pe observațiile $x_{t-1}, \ldots, x_{t-\tau}$.
   - Beneficiul imediat este că numărul de argumente (intrări) rămâne **constant** pentru $t > \tau$.
   - Acestea sunt numite **modele autoregresive** deoarece efectuează regresie pe ele însele.
   - Dacă această aproximare este precisă, se spune că secvența satisface **condiția Markov**.
2. **Modele Autoregresive Latente (Latent Autoregressive Models):**
   - Se menține un **rezumat ascuns** h_t al observațiilor trecute.
   - Predicția $\hat{x}_t$ se bazează pe $P(x_t|h_t)$, iar h_t este actualizată pe baza stării ascunse anterioare h_{t-1} și a intrării anterioare x_{t-1}.
   - Formula de actualizare este $h_t = g(h_{t-1}, x_{t-1})$.
   - Deoarece h_t nu este observată în date, aceste modele se numesc **modele autoregresive latente**. (Figura 2 arată vizual acest flux).

Proprietăți Secvențiale și Markov

- Pentru antrenare, se folosesc de obicei observațiile istorice pentru a prezice observația următoare.
- O presupunere comună este că, deși valorile specifice x_t se pot schimba, **dinamica secvenței în sine nu se va schimba**. Statisticienii numesc dinamica care nu se schimbă **staționară** (*stationary*).
- Estimarea întregii secvențe $P(x_1, \ldots, x_T)$ se obține prin produsul probabilităților condiționate:

$$P(x_1, \ldots, x_T) = {}_{t=1}^{T} P(x_t|x_{t-1}, \ldots, x_1)$$

- Dacă aproximarea autoregresivă este precisă, secvența satisface **condiția Markov**.
- În particular, dacă $\tau = 1$, avem un **model Markov de ordinul întâi**:

$$P(x_1, \ldots, x_T) = {}_{t=1}^{T} P(x_t|x_{t-1}) \text{ unde } P(x_1|x_0) = P(x_1)$$

Aplicații NLP și Preprocesarea Textului

- Modelele secvențiale sunt utile și pentru **obiecte discrete**, cum ar fi cuvintele. În acest caz, se folosește un model de **clasificare** (în loc de regresie) pentru a estima $P(x_t|x_{t-1}, \ldots, x_1)$.

- Textul este un exemplu popular de date secvențiale (o secvență de cuvinte sau caractere).
- **Scopul unui model de limbaj (*language model*)** este estimarea probabilității comune a secvenței $P(x_1, x_2, \ldots, x_T)$.
- Un model de limbaj ideal poate **genera text natural** prin extragerea unui *token* la momentul t din distribuția condiționată $P(x_t|x_{t-1}, \ldots, x_1)$.

Pași Comuni de Preprocesare a Textului:

1. Încărcarea textului ca șiruri (*strings*).
2. Împărțirea șirurilor în **token-uri** (cuvinte sau caractere). Un *token* este unitatea de bază.
3. Construirea unui **vocabular** (*dictionary*) care mapează *token*-urile textuale la **indici numerici** (începând de la 0).
4. Conversia textului în **secvențe de indici numerici**.

- Vocabularul este construit prin numărarea *token*-urilor unice din **corpus**.
- *Token*-urile rare sunt adesea eliminate.
- Se pot adăuga *token*-uri rezervate, cum ar fi `<unk>` (necunoscut), `<pad>` (padding), `<bos>` (început de secvență), și `<eos>` (sfârșit de secvență).

Modele N-Gram

- Aproximările bazate pe proprietatea Markov corespund la modelele n-gram:
  - **Unigram:** $P(x_1)P(x_2)P(x_3)P(x_4)$ (fără dependență).
  - **Bigram ($\tau = 1$):** $P(x_1)P(x_2|x_1)P(x_3|x_2)P(x_4|x_3)$.
  - **Trigram ($\tau = 2$):** $P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)P(x_4|x_2, x_3)$.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Modele Autoregresive Latente (h_t)

- **Analogie Simplificată (Buletinul de Știri):** Să presupunem că predicem prețul acțiunilor. Un model autoregresiv simplu folosește doar prețurile de ieri, alaltăieri, etc. Un model autoregresiv latent folosește un "Buletin de Știri" (h_t) pe care îl actualizează continuu. h_t rezumă toate evenimentele financiare, știrile politice și datele tehnice observate până la momentul t. Deși nu vedem direct h_t în datele de piață, acesta este starea internă a rețelei care rezumă contextul.

Concept Dificil: Direcția Temporală (Cauzalitate)

- **Context:** Deși teoretic se poate descompune probabilitatea secvenței în ordine inversă, în practică, viitorul nu poate influența trecutul.

- *Asta se întâmplă deoarece:* De obicei, suntem interesați de direcția **înainte** $P(x_{t+1}|x_t)$, deoarece dacă schimbăm x_t, putem influența x_{t+1}, dar nu invers. Acest lucru face ca modelarea în direcția **viitorului** să fie mai ușoară și mai relevantă.

Tokenizare și Vocabular

- **Context:** De ce trebuie să transformăm textul în indici numerici?
- *Asta se întâmplă deoarece:* Modelele neuronale lucrează exclusiv cu **intrări numerice** (vectori și matrice). *Token*-urile (cuvintele ca șiruri de caractere) trebuie mapate la numere (indici) pentru a putea fi prelucrate de straturile de intrare (adesea prin reprezentări de tip *embedding*).

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu RNN-urile (Lecția 4, Conversația Anterioară): Modelele autoregresive latente** sunt baza teoretică pentru majoritatea Rețelelor Neuronale Recurente (RNNs). Variabila de stare latentă h_t din modelele latente este exact **starea ascunsă** H_t din formulele RNN pe care le-am studiat anterior (unde H_t depinde de H_{t-1} și X_t).
- **Implicația Statică (*Stationary* Dynamics):** Presupunerea dinamicii staționare este crucială. Dacă, de exemplu, regulile pieței bursiere s-ar schimba complet mâine (o dinamică nouă), modelul antrenat pe datele vechi nu ar putea prezice noile dinamici.

---

5. EXERCITII (Calcul pe Modele Autoregresive Latente/RNN)

Voi folosi **Exercițiul 4** furnizat în sursa *04_Exercises.pdf*, deoarece acesta este un exercițiu complet care combină calculul de parametri și propagarea înainte specifică RNN, care ilustrează cel mai bine modelele autoregresive latente.

Exercițiul E.4: Calculul Propagării Înainte într-un RNN Autoregresiv Latent

Definiți un RNN pentru regresie având **2 intrări** ($d = 2$), o **stare ascunsă de dimensiunea 3** ($h = 3$), și **2 ieșiri** ($q = 2$). Se folosește o dimensiune a mini-batch-ului $n = 2$. Funcția de activare pentru starea ascunsă este $\phi(x) = \mathrm{ReLU}(x)$.

Calculați ieșirile O_1 și O_2 pentru doi pași de timp, utilizând următoarele date și ponderi (conform modelului din sursă):

- **Intrări:**

$$X_1 = [1 \quad 0 \ -1 \quad 1], \quad X_2 = [0 \quad 1\,1 \quad 0]$$

- **Starea inițială:** $H_0 = [0 \quad 0 \quad 0\,0 \quad 0 \quad 0] \in \mathbb{R}^{2\times 3}$
- **Ponderi și Bias-uri (generate):**

$$W_{xh} = [1 \quad 0 \quad 1\,0 \quad 1 \quad 0], \quad W_{hh} = [1 \quad 0 \quad -1\,0 \quad 1 \quad 0\,1 \quad -1 \quad 0], \quad b_h = [1 \quad 0 \quad -1]$$
$$W_{hq} = [1 \quad 0\,0 \quad 1\,0 \quad 1], \quad b_q = [1 \quad -1]$$

1. Specificați **numărul total de parametri reali** pentru strat.
2. Calculați H_1 și O_1.
3. Calculați H_2 și O_2.

Rezolvare E.4:

1. Calculul Total al Parametrilor

- W_{xh}: $d \times h = 2 \times 3 = 6$
- W_{hh}: $h \times h = 3 \times 3 = 9$
- b_h: $h = 3$
- W_{hq}: $h \times q = 3 \times 2 = 6$
- b_q: $q = 2$
- $$\text{Total parametri} = 6 + 9 + 3 + 6 + 2 = \mathbf{26}$$

2. Pasul de Timp $t = 1$

Calculul H_1:

$$H_1 = \phi(X_1 W_{xh} + H_0 W_{hh} + b_h)$$
$$H_1 = \phi\left([1 \quad 0 \; -1 \quad 1]\,[1 \quad 0 \quad 1\,0 \quad 1 \quad 0] + [0 \quad 0 \quad 0\,0 \quad 0 \quad 0] + [1 \quad 0 \quad -1\,1 \quad 0 \quad -1]\right)$$
$$H_1 = \phi\left([1 \quad 0 \quad 1 \; -1 \quad 1 \quad -1] + [1 \quad 0 \quad -1\,1 \quad 0 \quad -1]\right) = \phi\left([2 \quad 0 \quad 0\,0 \quad 1 \quad -2]\right)$$
$$H_1 = [2 \quad 0 \quad 0\,0 \quad 1 \quad 0] \in \mathbb{R}^{2\times 3}$$

(Aplicând ReLU)

Calculul O_1:

$$O_1 = H_1 W_{hq} + b_q$$
$$O_1 = [2 \quad 0 \quad 0\,0 \quad 1 \quad 0]\,[1 \quad 0\,0 \quad 1\,0 \quad 1] + [1 \quad -1\,1 \quad -1]$$
$$O_1 = [2 \quad 0\,0 \quad 1] + [1 \quad -1\,1 \quad -1] = [3 \quad -1\,1 \quad 0] \in \mathbb{R}^{2\times 2}$$

3. Pasul de Timp $t = 2$

Calculul H_2:

$$H_2 = \phi(X_2 W_{xh} + H_1 W_{hh} + b_h)$$
$$H_2 = \phi\left([0 \quad 1\,1 \quad 0]\,[1 \quad 0 \quad 1\,0 \quad 1 \quad 0] + [2 \quad 0 \quad 0\,0 \quad 1 \quad 0]\,[1 \quad 0 \quad -1\,0 \quad 1 \quad 0\,1 \quad -1 \quad 0] + [1 \quad 0\right.$$

$$H_2 = \phi\left([0 \quad 1 \quad 0\,1 \quad 0 \quad 1] + [2 \quad 0 \quad -2\,0 \quad 1 \quad 0] + [1 \quad 0 \quad -1\,1 \quad 0 \quad -1]\right)$$

$$H_2 = \phi\left([3 \quad 1 \quad -3\,2 \quad 1 \quad 0]\right)$$

$$H_2 = [3 \quad 1 \quad 0\,2 \quad 1 \quad 0] \in \mathbb{R}^{2\times 3}$$

(Aplicând ReLU)

Calculul O_2:

$$O_2 = H_2 W_{hq} + b_q$$

$$O_2 = [3 \quad 1 \quad 0\,2 \quad 1 \quad 0]\,[1 \quad 0\,0 \quad 1\,0 \quad 1] + [1 \quad -1\,1 \quad -1]$$

$$O_2 = [3 \quad 1\,2 \quad 1] + [1 \quad -1\,1 \quad -1] = [4 \quad 0\,3 \quad 0] \in \mathbb{R}^{2\times 2}$$

4.2

Bine ați venit la lecția **4.2 Rețele Neuronale Recurente (Recurrent Neural Networks - RNNs)**. Această secțiune este esențială pentru a înțelege cum implementăm de fapt modelul autoregresiv latent, definind structura matematică a rețelelor recurente, dar și provocările majore de antrenare (explozia/dispariția gradientului) și soluțiile practice.

Ghid de Studiu Complet: 4.2 Rețele Neuronale Recurente

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 4.2 detaliază structura matematică a RNN-urilor, care sunt **rețele neuronale cu stări ascunse**. RNN-urile oferă o soluție elegantă pentru limitările modelelor n-gram, unde numărul de parametri crește exponențial cu lungimea dependenței ($|\mathcal{V}|^n$). Prin utilizarea unei **variabile de stare latentă (h_{t-1})** (cunoscută și ca stare ascunsă) pentru a stoca informațiile secvențiale trecute, RNN-urile aproximează dependența lungă $P(x_t|x_{t-1},\ldots,x_1) \approx P(x_t|h_{t-1})$. Deoarece calculul stării ascunse utilizează aceeași definiție la fiecare pas de timp, acesta este un calcul **recurent**. Un avantaj major este că **costul de parametrizare al unui RNN nu crește odată cu numărul de pași de timp**. Totuși, secvențele lungi generează lanțuri lungi de produse matriciale în timpul *backpropagation*-ului, ducând la **instabilitate numerică** (explozia sau dispariția gradientului). Această problemă este atenuată de tehnici precum **Gradient Clipping**.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Modelul Stării Ascunse Latente:** RNN-urile folosesc variabila de stare ascunsă h_{t-1} pentru a rezuma istoria secvenței. Starea ascunsă curentă (H_t) este o funcție neliniară de inputul curent (X_t) și starea ascunsă anterioară (H_{t-1}).
2. **Formulă de Recurență:** Calculul stării ascunse este dat de $H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$.

3. **Partajarea Parametrilor:** Spre deosebire de modelele care ar avea parametri unici pentru fiecare pas de timp, RNN-urile folosesc aceleași matrici de ponderi (W_{xh}, W_{hh}, W_{hq}) la momente diferite. Acest lucru menține costul parametrilor constant.
4. **Instabilitatea Gradientului:** Antrenarea pe secvențe lungi rezultă în instabilitate numerică în timpul *backpropagation*-ului, deoarece gradientul este un lanț de produse matriciale de lungime $O(T)$.
5. **Perplexitatea (Măsurarea Calității):** Calitatea unui model de limbaj este măsurată prin **perplexitate**, care este exponențialul pierderii de entropie încrucișată medie pe secvență.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4.2 Rețele Neuronale Recurente (RNNs)

De la N-Gram la Stări Ascunse Latente

- Modelele n-gram au dezavantajul că, pentru a crește efectul cuvintelor anterioare momentului t, trebuie mărit n, ceea ce duce la o creștere exponențială a numărului de parametri ($|\mathcal{V}|^n$).
- RNN-urile preferă să modeleze probabilitatea condiționată $P(x_t|x_{t-1}, \ldots, x_1)$ printr-un model cu variabilă latentă: $P(x_t|x_{t-1}, \ldots, x_1) \approx P(x_t|h_{t-1})$.
- h_{t-1} este o **stare ascunsă** (*hidden state* sau *hidden variable*) care stochează informația secvențială până la momentul $t-1$.

Definirea Stării Ascunse (H_t)

- Starea ascunsă la momentul t se calculează pe baza intrării curente x_t și a stării ascunse anterioare h_{t-1}: $h_t = f(x_t, h_{t-1})$.
- **Atenție la Terminologie:** Straturile ascunse (*hidden layers*) sunt straturi ascunse privirii pe calea de la intrare la ieșire. Stările ascunse (*hidden states*) sunt intrări calculate pe baza datelor din pașii de timp anteriori.

Formula de Recurență (EC. 3)

- Intrarea este un mini-batch $X_t \in \mathbb{R}^{n \times d}$, unde n este mărimea batch-ului și d este dimensiunea intrărilor.
- Starea ascunsă $H_t \in \mathbb{R}^{n \times h}$ se calculează ca:

$$\mathbf{H_t} = \phi(\mathbf{X_t W_{xh}} + \mathbf{H_{t-1} W_{hh}} + \mathbf{b_h}) \quad (3)$$

  - $W_{xh} \in \mathbb{R}^{d \times h}$ (ponderi Intrare $\rightarrow$ Stare Ascunsă)
  - $W_{hh} \in \mathbb{R}^{h \times h}$ (ponderi Stare Ascunsă $\rightarrow$ Stare Ascunsă)
  - $b_h \in \mathbb{R}^{1 \times h}$ (bias-ul stratului ascuns)
- Această formulă adaugă termenul $H_{t-1}W_{hh}$ față de un MLP.

- Calculul $X_t W_{xh} + H_{t-1} W_{hh}$ este echivalent cu înmulțirea matricială dintre concatenarea lui X_t și H_{t-1} și concatenarea lui W_{xh} și W_{hh}.
- Stratul care efectuează calculul (3) se numește **strat recurent** (*recurrent layer*).

Stratul de ieșire (O_t)

- Ieșirea $O_t \in \mathbb{R}^{n \times q}$ este similară cu cea din MLP:

$$\mathbf{O_t} = \mathbf{H_t W_h} + \mathbf{b}$$

  - $W_{hq} \in \mathbb{R}^{h \times q}$ (ponderi Stare Ascunsă $\rightarrow$ Ieșire)
  - $b_q \in \mathbb{R}^{1 \times q}$ (bias-ul stratului de ieșire)

Costul Parametrizării și Stabilitatea

- RNN-urile folosesc **aceiași parametri de model** la momente diferite. Prin urmare, costul de parametrizare nu crește odată cu lungimea secvenței.
- **Instabilitate:** Antrenarea unei secvențe de lungime T necesită calculul gradientului pe T pași de timp (lanț de produse matriciale de lungime $O(T)$). Când T este mare, acest lucru duce la instabilitate numerică (explozia sau dispariția gradientului).

Gradient Clipping (Tăierea Gradientului)

- O soluție populară pentru a stabiliza antrenarea este **Gradient Clipping**.
- Aceasta proiectează vectorul gradient g înapoi într-o bilă de rază dată θ.
- Formula de tăiere: $g \leftarrow \min\left(1, \frac{\theta}{|g|}\right) g$.
- Prin această metodă, norma gradientului nu depășește niciodată θ.
- Gradient Clipping oferă un anumit grad de **robustețe** modelului, limitând influența unui mini-batch asupra vectorului de parametri.

Măsurarea Calității Modelului de Limbaj (Perplexitatea)

- Calitatea modelului de limbaj se măsoară prin cât de **surprinzător** este textul. Un model bun prezice cu acuratețe ridicată *token*-urile următoare.
- Perplexitatea este exponențialul pierderii de entropie încrucișată medie pe secvență:

$$\text{Perplexitatea} = \exp\left(-\frac{1}{n}\sum_{t=1}^{n} \log P(x_t | x_{t-1}, \ldots, x_1)\right)$$

- **Interpretare Perplexitate:**
  - **Cel mai bun caz:** Modelul prezice perfect, probabilitatea *token*-ului corect este 1. Perplexitatea este **1**.
  - **Cel mai slab caz:** Modelul prezice probabilitatea 0 pentru *token*-ul corect. Perplexitatea este **infinit pozitiv**.
  - **Baseline:** Modelul prezice o distribuție uniformă peste întregul vocabular. Perplexitatea este egală cu **numărul de *token*-uri unice din vocabular**. Un model

util trebuie să bată această limită superioară non-trivială.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Stări Ascunse vs. Straturi Ascunse

- **Explicație Simplificată (ELI5): Straturile ascunse** sunt cutii negre în rețeaua ta. **Stările ascunse** (H_t) sunt ca o "memorie" care iese dintr-o cutie la momentul t și re-intră în aceeași cutie (sau următoarea) la momentul $t+1$. Stările ascunse sunt **intrări** care depind de istorie, nu doar simple ieșiri ascunse.

Concept Dificil: Gradient Clipping

- **Analogie (Limită de Viteză pe Drumul Optimizării):** Optimizarea (căutarea minimului funcției de pierdere) este ca și cum ai coborî pe un deal. Gradientul îți spune direcția. Dacă dealul devine brusc foarte abrupt (gradient mare), un pas mare de actualizare te poate arunca departe de soluție (explozia gradientului, sau *overshooting*). Gradient Clipping impune o **limită de viteză** (θ) pentru a preveni pașii de actualizare excesiv de mari, menținând procesul pe drumul cel bun, chiar dacă înseamnă că progresul este puțin mai lent.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Cost de Calcul vs. Parametri):**
  - **CAUZĂ:** Partajarea ponderilor (W_{xh}, W_{hh}, etc.) pe parcursul timpului.
  - **EFECT:** Numărul total de parametri rămâne constant, indiferent de lungimea secvenței T. Însă, costul de calcul în timpul antrenării (*runtime*) crește liniar cu T, deoarece propagarea înainte și înapoi trebuie efectuată la fiecare pas de timp.
- **Conexiunea cu Secțiunea 4.1:** RNN-ul este realizarea computațională a modelului autoregresiv latent, unde variabila latentă h_{t-1} este implementată ca starea ascunsă H_{t-1}.

---

5. EXERCITII (Calcul pe Arhitectura RNN)

Voi genera un exercițiu de calcul bazat pe structura RNN definită în secțiunea 4.2 și un exercițiu conceptual despre stabilitate.

Exercițiul R.5: Calculul Propagării Înainte (RNN cu Stare Ascunsă Mică)

Definiți un RNN pentru regresie cu **2 intrări** ($d = 2$), o **stare ascunsă de dimensiunea 2** ($h = 2$), și **1 ieșire** ($q = 1$). Funcția de activare pentru starea ascunsă este $\phi(x) = \mathrm{ReLU}(x)$. Mărimea mini-batch-ului $n = 2$.

Calculați H_1 și O_1 pentru primul pas de timp, folosind următoarele intrări și ponderi:

- **Intrări:**

$$X_1 = [2 \quad 0\,0 \quad 4] \in \mathbb{R}^{2\times 2}$$

- **Starea inițială:** $H_0 = [1 \quad 0\,0 \quad 1] \in \mathbb{R}^{2\times 2}$
- **Ponderi și Bias-uri:**

$$W_{xh} = [1 \quad 0\,0 \quad 1], \quad W_{hh} = [0 \quad 1\,1 \quad 0], \quad b_h = [-1 \quad 0]$$
$$W_{hq} = [1\,0], \quad b_q = [1]$$

1. Calculați H_1.
2. Calculați O_1.
3. Specificați **numărul total de parametri reali** (greutăți și bias-uri) pentru strat.

Rezolvare R.5:

3. Calculul Total al Parametrilor

- W_{xh}: $d \times h = 2 \times 2 = 4$
- W_{hh}: $h \times h = 2 \times 2 = 4$
- b_h: $h = 2$
- W_{hq}: $h \times q = 2 \times 1 = 2$
- b_q: $q = 1$
- $$\text{Total parametri} = 4 + 4 + 2 + 2 + 1 = \mathbf{13}$$

1. Pasul de Timp $t = 1$ (Calculul H_1)

$$H_1 = \phi(X_1 W_{xh} + H_0 W_{hh} + b_h)$$

$$X_1 W_{xh} = [2 \quad 0\,0 \quad 4]\,[1 \quad 0\,0 \quad 1] = [2 \quad 0\,0 \quad 4]$$

$$H_0 W_{hh} = [1 \quad 0\,0 \quad 1]\,[0 \quad 1\,1 \quad 0] = [0 \quad 1\,1 \quad 0]$$

$$\text{Suma Afină} = [2 \quad 0\,0 \quad 4] + [0 \quad 1\,1 \quad 0] + [-1 \quad 0 \; -1 \quad 0]$$

$$\text{Suma Afină} = [1 \quad 1\,0 \quad 4]$$

$$H_1 = \phi\left([1 \quad 1\,0 \quad 4]\right) = [1 \quad 1\,0 \quad 4]$$

(Aplicând ReLU, deoarece toate valorile sunt 0)

2. Calculul O_1

$$O_1 = H_1 W_{hq} + b_q$$

$$O_1 = [1 \quad 1\ 0 \quad 4]\,[1\ 0] + [1\ 1]$$

$$O_1 = [(1 \cdot 1) + (1 \cdot 0)\ (0 \cdot 1) + (4 \cdot 0)] + [1\ 1] = [1\ 0] + [1\ 1] = [2\ 1]$$

Exercițiul R.6: Perplexitatea și Gradient Clipping (Conceptual)

1. Dacă un model de limbaj are un vocabular de 100 de *token*-uri, care este **valoarea maximă a perplexității** la *baseline*? Explicați scurt ce indică această valoare.
2. Un antrenament RNN raportează o normă de gradient ($|g|$) de 25.0 la un pas de timp. Dacă raza de tăiere (clipping radius) este setată la $\theta = 5.0$, calculați noua normă a gradientului (g').

Rezolvare R.6:

1. **Perplexitatea la Baseline:** Valoarea maximă a perplexității la *baseline* este egală cu numărul de *token*-uri unice din vocabular.
   - **Valoarea: 100**.
   - **Indicație:** Această valoare reprezintă performanța unui model care prezice o **distribuție uniformă** a probabilităților peste toate *token*-urile vocabularului. Un model util trebuie să obțină o perplexitate **mai mică** de 100 pentru a fi considerat mai bun decât o ghicire aleatorie.
2. **Calculul Gradient Clipping:** Se folosește formula $g' = \min\left(1, \frac{\theta}{|g|}\right) g$:

   $$\text{Factor de scalare} = \min\left(1, \frac{5.0}{25.0}\right) = \min(1, 0.2) = 0.2$$

   Noua normă a gradientului este scalată de acest factor:

   $$|g'| = |g| \times \text{Factor de scalare} = 25.0 \times 0.2 = \mathbf{5.0}$$

   (Noul gradient este proiectat înapoi la raza de tăiere $\theta = 5.0$).

4.3

Această lecție, **4.3 Gated Recurrent Units (GRU)**, reprezintă o evoluție crucială a Rețelelor Neuronale Recurente (RNNs). Deși RNN-urile standard pot gestiona date secvențiale, ele se confruntă cu probleme severe de **instabilitate numerică** (explozia sau dispariția gradientului), în special pe secvențe lungi. GRU, împreună cu LSTM (Long Short-Term Memory), oferă o soluție prin introducerea unor mecanisme inteligente de **gating** (porți), care permit rețelei să învețe în mod selectiv *când* să rețină informația trecută și *când* să o ignore sau să o reseteze, mitigând astfel problema dependențelor pe termen lung.

Ghid de Studiu Complet: 4.3 Gated Recurrent Units (GRU)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Rețelele neuronale recurente de bază (*vanilla RNNs*), deși au introdus conceptul de stare ascunsă pentru date secvențiale, nu sunt suficiente pentru o gamă largă de probleme din practica modernă din cauza **instabilității numerice**. Unitatea Recurentă Poartă (GRU), propusă în **2014**, este un design mai sofisticat care face parte din clasa RNN-urilor cu porți (*gated RNNs*), fiind mai ușor de calculat și adesea oferind performanțe comparabile cu cele ale LSTM (propus în **1997**). Distincția cheie este că GRU introduce mecanisme de **resetare (R_t)** și **actualizare (Z_t)** care sunt învățate de model. Aceste porți permit modelului să decidă dacă să păstreze informația importantă (chiar și pe distanțe mari de timp), să sară peste observațiile irelevante sau să reseteze starea internă, rezolvând astfel problemele practice ale dependențelor pe termen lung și ale gradientului evanescent.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Necesitatea Gating-ului:** GRU a fost introdus pentru a rezolva problemele de **instabilitate numerică** (gradient *vanishing* sau *exploding*) ale RNN-urilor simple.
2. **Mecanismul de Control:** GRU are mecanisme dedicate pentru a controla *când* starea ascunsă trebuie **actualizată (Z_t)** și *când* trebuie **resetată (R_t)**.
3. **Funcția Porților:** Porțile (R_t și Z_t) sunt calculate ca ieșiri ale unor straturi complet conectate cu funcția de activare **Sigmoid (σ)**, forțând valorile în intervalul $(0, 1)$ pentru a realiza combinații convexe.
4. **Rolul R_t și Z_t:** Poarta de **resetare (R_t)** ajută la capturarea dependențelor pe **termen scurt**. Poarta de **actualizare (Z_t)** ajută la capturarea dependențelor pe **termen lung** (permițând rețelei să rețină starea veche, ignorând *timestep*-ul curent, dacă $Z_t \approx 1$).
5. **Starea Finală:** Starea ascunsă finală (H_t) este o **combinație convexă** a stării ascunse vechi (H_{t-1}) și a noii stări candidate ($\tilde{H}_t$), ponderată de poarta de actualizare (Z_t).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4.3 Gated recurrent units (GRU)

Motivația și Contextul Istoric

- Deși s-au introdus bazele RNN-urilor, tehnicile simple nu sunt suficiente pentru o gamă largă de probleme de învățare secvențială.
- O problemă notabilă în practică este **instabilitatea numerică** a RNN-urilor (gradient *vanishing* sau *exploding*), care apare din cauza lanțurilor lungi de produse matriciale în timpul *backpropagation*-ului.

- Modelele cu porți (*gated RNNs*), precum GRU și LSTM, sunt mult mai comune în practică și atenuează această problemă.
- **LSTM** (Long Short-Term Memory) a fost propusă în **1997** și este cea mai veche metodă.
- **GRU** (Gated Recurrent Unit), propusă în **2014**, este o variantă mai ușoară care oferă adesea performanțe comparabile și este semnificativ mai rapidă la calcul.

Probleme practice adresate de Gating:

1. **Păstrarea Informației Vitale:** Necesitatea de a stoca informații timpurii foarte semnificative (ex: o sumă de control/checksum) și de a-i atribui un gradient mare, deoarece afectează toate observațiile ulterioare.
2. **Sărirea (Skipping) Observațiilor Irelevante:** Necesitatea de a ignora *token*-uri care nu poartă informații pertinente (ex: cod HTML auxiliar la parsarea unei pagini web).
3. **Resetarea Stării Interne:** Necesitatea de a reseta reprezentarea stării interne latente la o **întrerupere logică** în secvență (ex: tranziția între capitole într-o carte sau schimbarea pieței la acțiuni).

Mecanismul Porților (R_t și Z_t)

- GRU-urile suportă *gating*-ul stării ascunse, având mecanisme dedicate pentru **actualizare** și **resetare**.
- Se introduc **poarta de resetare (R_t)** și **poarta de actualizare (Z_t)**.
- Acestea sunt vectori cu intrări în intervalul $(0, 1)$ pentru a permite **combinații convexe**.

Calculul Porților (Ecuația Porților):

- Porțile sunt date de **două straturi complet conectate** cu funcția de activare **Sigmoid (σ)**.
- Intrările sunt $X_t \in \mathbb{R}^{n \times d}$ și $H_{t-1} \in \mathbb{R}^{n \times h}$.
- $R_t \in \mathbb{R}^{n \times h}$ și $Z_t \in \mathbb{R}^{n \times h}$.
- $$\mathbf{_t} = \sigma(\mathbf{X_t W_x} + \mathbf{H_{t-1} W_h} + \mathbf{b})$$
- $$\mathbf{Z_t} = \sigma(\mathbf{X_t W_{xz}} + \mathbf{H_{t-1} W_{hz}} + \mathbf{b_z})$$
- Funcția σ transformă intrările în intervalul $(0, 1)$.

Starea Ascunsă Candidată ($\tilde{H}_t$):

- Această etapă integrează poarta de resetare R_t cu mecanismul standard de actualizare a stării latente (Ecuația 3 a RNN).
- Se utilizează **funcția** $\tanh$ pentru neliniaritate, asigurând că valorile rămân în intervalul $(-1, 1)$.
- $$\mathbf{\tilde{H}_t} = \tanh(\mathbf{X_t W_{xh}} + (\mathbf{_t} \odot \mathbf{H_{t-1}})\mathbf{W_{hh}} + \mathbf{b_h}) \quad (5)$$

- Simbolul $\odot$ este operatorul de **produs Hadamard** (element-wise).
- **Efectul R_t:**
  - Dacă $R_t \approx 1$, se recuperează un RNN simplu (*vanilla RNN*).
  - Dacă $R_t \approx 0$, termenul $R_t \odot H_{t-1}$ este aproape zero, iar starea ascunsă anterioară este **resetată** la *default*. $\tilde{H}_t$ devine rezultatul unui MLP care folosește doar X_t ca input.

Actualizarea Finală a Stării Ascunse (H_t):

- Poarta de actualizare Z_t determină în ce măsură noua stare ascunsă H_t este doar o copie a stării vechi H_{t-1} sau utilizează noua stare candidată $\tilde{H}_t$.
- Se folosesc **combinații convexe element-wise**:

$$\mathbf{H_t} = \mathbf{Z_t} \odot \mathbf{H_{t-1}} + (\mathbf{1} - \mathbf{Z_t}) \odot \tilde{\mathbf{H}}_\mathbf{t}$$

- **Efectul Z_t:**
  - Dacă $Z_t \approx 1$, $H_t \approx H_{t-1}$. Informația din X_t este ignorată, **sărind efectiv peste pasul de timp** t în lanțul de dependență. Aceasta ajută la capturarea dependențelor pe **termen lung**.
  - Dacă $Z_t \approx 0$, $H_t \approx \tilde{H}_t$. Noua stare latentă absoarbe informația din pasul curent.

Extinderi Arhitecturale

- Arhitectura RNN cu un singur strat ascuns poate fi extinsă la **arhitecturi profunde** cu multiple straturi ascunse, extinderi care sunt adoptate frecvent în rețelele recurente moderne.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Produsul Hadamard ($\odot$)

- **Explicație Simplificată (Comutator de Lumină):** Produsul Hadamard (element-wise) este esențial pentru *gating*. Dacă avem un vector de informații vechi (H_{t-1}) și un vector de porți (R_t) cu valori între 0 și 1, $R_t \odot H_{t-1}$ acționează ca un **variator de intensitate (dimmer)**. O intrare în R_t aproape de 0 "stinge" acea porțiune din starea veche (uită informația), iar o intrare aproape de 1 o "lasă să treacă" (reține informația).

Concept Dificil: Rolul Dual al Porților

- **Analogie (Librarul și Portarul):** Imaginați-vă că H_{t-1} este conținutul unei cărți (memoria veche), iar $\tilde{H}_t$ este o pagină nouă.
  - **Poarta de Resetare (R_t)** (Portarul): Decide cât de mult din informația veche din cartea H_{t-1} este **relevantă** pentru a scrie noua pagină ($\tilde{H}_t$). Dacă $R_t \approx 0$,

conținutul vechi este inutil pentru noua pagină. (Ajută la dependențe pe termen scurt).

- **Poarta de Actualizare (Z_t)** (Librarul): Decide dacă noua pagină ($\tilde{H}_t$) este suficient de importantă pentru a înlocui vechea stare sau dacă trebuie să **păstrezi** pur și simplu vechea carte (H_{t-1}). Dacă $Z_t \approx 1$, se păstrează H_{t-1}, permițând informației timpurii să traverseze un număr mare de pași de timp. (Ajută la dependențe pe termen lung).

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu ResNet (Secțiunea 3.10):** Mecanismul de actualizare $H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$ seamănă conceptual cu **conexiunea reziduală** (shortcut connection) a ResNet. Dacă $Z_t \approx 1$, informația din H_{t-1} trece aproape neschimbată (funcția identitate), oferind o cale directă pentru fluxul gradientului și prevenind *vanishing gradient*.
- **Implicația asupra Memoriei:** Prin capacitatea porții de actualizare de a se menține aproape de 1 pentru o sub-secvență lungă, informația critică de la începutul secvenței poate fi **reținută și propagată** până la sfârșit, indiferent de lungimea secvenței.

---

5. EXERCITII (Calcul pe Arhitectura GRU)

Pentru a exersa mecanismul GRU, vom genera un exercițiu simplificat bazat pe formulele porților și stării candidate.

Exercițiul G.1: Calculul Porților și Stării Candidate (GRU Scalar)

Considerați un singur pas de timp ($t = 1$) într-un GRU simplificat (dimensiune scalară $d = 1, h = 1, n = 1$).

- Intrare: $X_1 = 5$.
- Stare ascunsă anterioară: $H_0 = 10$.

Presupuneți că toate ponderile și bias-urile sunt scalari, având valorile:

- Ponderi Poarta Resetare: $W_{xr} = 1, W_{hr} = -0.5, b_r = 0$.
- Ponderi Poarta Actualizare: $W_{xz} = 0, W_{hz} = 0.5, b_z = 0$.
- Ponderi Stare Candidată: $W_{xh} = 1, W_{hh} = 0.1, b_h = -1$.

Simplificări Funcții de Activare:

- $\sigma(x) = 1/(1 + {}^{-x})$. Pentru simplitate, presupunem: $\sigma(2.5) \approx 0.9$ și $\sigma(5) \approx 1.0$.
- $\tanh(x)$. Presupunem: $\tanh(4) \approx 1.0$.

1. Calculați Poarta de Resetare R_1.
2. Calculați Poarta de Actualizare Z_1.
3. Calculați Starea Ascunsă Candidată $\tilde{H}_1$.

Rezolvare G.1:

1. Calculul Poartei de Resetare (R_1)

$$R_1 = \sigma(X_1 W_{xr} + H_0 W_{hr} + b_r)$$

$$R_1 = \sigma(5 \cdot 1 + 10 \cdot (-0.5) + 0)$$

$$R_1 = \sigma(5 - 5) = \sigma(0) = 0.5$$

$$_{\mathbf{1}} = \mathbf{0.5}$$

2. Calculul Poartei de Actualizare (Z_1)

$$Z_1 = \sigma(X_1 W_{xz} + H_0 W_{hz} + b_z)$$

$$Z_1 = \sigma(5 \cdot 0 + 10 \cdot 0.5 + 0)$$

$$Z_1 = \sigma(5) \approx \mathbf{1.0}$$

3. Calculul Stării Ascunse Candidate ($\tilde{H}_1$)

$$\tilde{H}_1 = \tanh(X_1 W_{xh} + (R_1 \odot H_0) W_{hh} + b_h)$$

Termenul recurent: $R_1 \odot H_0 = 0.5 \cdot 10 = 5$.

$$\tilde{H}_1 = \tanh(5 \cdot 1 + 5 \cdot 0.1 - 1)$$

$$\tilde{H}_1 = \tanh(5 + 0.5 - 1) = \tanh(4.5)$$

Presupunând $\tanh(4.5) \approx 1.0$:

$$\mathbf{\tilde{H}_1 \approx 1.0}$$

4. Calculul Stării Ascunse Finale (H_1) (Bonus)

$$H_1 = Z_1 \odot H_0 + (1 - Z_1) \odot \tilde{H}_1$$

Deoarece $Z_1 \approx 1$:

$$H_1 \approx 1.0 \cdot 10 + (1 - 1.0) \cdot 1.0 = 10 + 0 = \mathbf{10}$$

(Observație: $Z_1 \approx 1$ a cauzat ignorarea stării candidate $\tilde{H}_1$ și reținerea stării vechi H_0, conform teoriei dependențelor pe termen lung.)

Exercițiul G.2: Analiza Arhitecturală GRU (Număr Parametri)

Calculați numărul total de parametri reali (greutăți și bias-uri) conținuți într-un singur strat GRU cu următoarele dimensiuni:

- Intrare $d = 10$.

- Stare ascunsă $h = 20$.

Rezolvare G.2:

Parametrii GRU necesită ponderi și bias-uri pentru trei calcule majore (Reset, Update, Candidate), unde fiecare necesită o transformare a intrării ($W_{x\bullet}$) și o transformare recurentă ($W_{h\bullet}$).

1. **Poarta de Resetare (R_t):**
   - $W_{xr}(d \times h)$: $10 \times 20 = 200$
   - $W_{hr}(h \times h)$: $20 \times 20 = 400$
   - $b_r(1 \times h)$: 20

$$\text{Total } R_t = 620$$

2. **Poarta de Actualizare (Z_t):**
   - $W_{xz}(d \times h)$: $10 \times 20 = 200$
   - $W_{hz}(h \times h)$: $20 \times 20 = 400$
   - $b_z(1 \times h)$: 20

$$\text{Total } Z_t = 620$$

3. **Starea Candidată ($\tilde{H}_t$):**
   - $W_{xh}(d \times h)$: $10 \times 20 = 200$
   - $W_{hh}(h \times h)$: $20 \times 20 = 400$
   - $b_h(1 \times h)$: 20

$$\text{Total } \tilde{H}_t = 620$$

$$\text{Total parametri GRU} = 620 + 620 + 620 = \mathbf{1860}$$

4.4

Suntem la o etapă critică a studiului nostru: **4.4 Long Short-Term Memory (LSTM)**. Această rețea nu este doar o evoluție, ci una dintre cele mai influente invenții din istoria *deep learning*-ului secvențial, oferind o soluție robustă pentru gestionarea memoriei pe termen lung.

Ghid de Studiu Complet: 4.4 Long Short-Term Memory (LSTM)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 4.4 introduce **Long Short-Term Memory (LSTM)**, o arhitectură recurentă care răspunde provocării de a adresa **conservarea informației pe termen lung** și **sărirea**

intrărilor pe termen scurt în modelele cu variabile latente. LSTM este una dintre cele mai vechi abordări pentru această problemă, precedând GRU cu aproape două decenii, deși LSTMs au un design ușor mai complex decât GRU. Designul LSTM este inspirat de porțile logice ale unui computer. Soluția cheie a LSTM este introducerea unei **celule de memorie (C_t)** separată de starea ascunsă, care acționează ca un rezervor pentru informațiile critice. Această celulă este controlată de un set de **trei porți** specializate.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Originea:** LSTM este una dintre cele mai timpurii și mai complexe soluții pentru gestionarea dependențelor pe termen lung, precedând GRU cu aproape două decenii.
2. **Celula de Memorie (C_t):** LSTM introduce o **celulă de memorie** (sau *cell* pe scurt) care are aceeași formă ca starea ascunsă și este concepută să înregistreze informații suplimentare.
3. **Mecanismul de Control:** Celula de memorie este controlată de trei porți dedicate: **Poarta de Intrare (I_t)**, **Poarta de Uitarea (F_t)** și **Poarta de Ieșire (O_t)**.
4. **Scopul Cheie:** Designul este introdus pentru a atenua problema **dispariției gradientului** (*vanishing gradient problem*) și pentru a capta mai bine dependențele pe distanțe lungi în secvențe.
5. **Actualizarea Celulei:** Celula de memorie este actualizată printr-o combinație element-wise a memoriei vechi (ponderată de Poarta de Uitarea F_t) și a stării candidate $\tilde{C}_t$ (ponderată de Poarta de Intrare I_t).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4.4 Long Short-Term Memory (LSTM)

Arhitectura și Componentele Cheie

- Provocarea de a adresa conservarea informației pe termen lung și ignorarea intrărilor pe termen scurt în modelele cu variabile latente există de mult timp.
- LSTM împărtășește multe proprietăți cu GRU, având aceeași motivație de bază: să decidă **când să rețină și când să ignore intrările** în starea ascunsă, printr-un mecanism dedicat.
- Designul LSTM este ușor mai complex decât cel al GRU.

Celula de Memorie și Porțile

- LSTM introduce o **celulă de memorie (C_t)** care are aceeași formă ca starea ascunsă (H_t), fiind proiectată pentru a înregistra informații suplimentare.
- Pentru a controla celula de memorie, sunt necesare un număr de porți:
    - **Poarta de Ieșire (O_t):** Necesară pentru a citi intrările din celula de memorie.

- **Poarta de Intrare (I_t):** Necesară pentru a decide când să citească date noi în celulă.
- **Poarta de Uitarea (F_t):** Un mecanism pentru a reseta conținutul celulei.

Calculul Porților

- Similar cu GRU, datele introduse în porțile LSTM sunt **intrarea la pasul de timp curent (X_t)** și **starea ascunsă a pasului de timp anterior (H_{t-1})**.
- Porțile sunt procesate de trei straturi complet conectate cu o funcție de activare **Sigmoid (σ)**.
- Ca rezultat, valorile celor trei porți (I_t, F_t, O_t) sunt în intervalul $(0, 1)$.
- **Intrarea $X_t \in \mathbb{R}^{n \times d}$, starea ascunsă anterioară $H_{t-1} \in \mathbb{R}^{n \times h}$**. Porțile $I_t, F_t, O_t \in \mathbb{R}^{n \times h}$.
- **Formulele Porților (Ec. 4.4.1):**

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

Celula de Memorie Candidată ($\tilde{C}_t$)

- Se introduce celula de memorie candidată $\tilde{C}_t \in \mathbb{R}^{n \times h}$.
- Calculul său este similar cu cel al celor trei porți, dar folosește funcția de activare $\tanh$, având un interval de valori de $(-1, 1)$.
- **Formula Celulei Candidate (Ec. 4.4.2):**

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

Actualizarea Celulei de Memorie (C_t)

- Actualizarea utilizează element-wise multiplication.
- **Poarta de Uitarea (F_t)** determină cât de mult din conținutul vechi al celulei C_{t-1} este reținut.
- **Poarta de Intrare (I_t)** determină cât de multă informație nouă din celula candidată $\tilde{C}_t$ este preluată.
- **Formula de Actualizare (Ec. 4.4.3):**

$$\mathbf{C_t} =_{\mathbf{t}} \odot \mathbf{C_{t-1}} +_{\mathbf{t}} \odot \mathbf{\tilde{C}_t}$$

- Dacă Poarta de Uitarea este aproape de 1, iar Poarta de Intrare este aproape de 0, conținutul vechi al celulei C_{t-1} este salvat și propagat la pasul de timp curent.

Calculul Stării Ascunse Finale (H_t)

- Starea ascunsă finală $H_t \in \mathbb{R}^{n \times h}$ este o versiune controlată (*gated*) a funcției $\tanh$ a celulei de memorie curente C_t.

- Aceasta asigură că valorile H_t se află întotdeauna în intervalul $(-1, 1)$.
- **Formula Stării Ascunse (Ec. 4.4.4):**

$$\mathbf{H_t} = \mathbf{O_t} \odot \tanh(\mathbf{C_t})$$

- Dacă Poarta de Ieșire (O_t) se apropie de 1, informația din memorie este transmisă predictorului; dacă O_t este aproape de 0, informația este reținută doar în celula de memorie, fără procesare ulterioară.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Celula de Memorie (C_t) vs. Starea Ascunsă (H_t)

- **Analogie Simplificată (Arhiva și Ecranul):**
  - **Celula de Memorie (C_t):** Este ca o arhivă sigilată sau un registru principal. Conține toată informația istorică importantă, neschimbată de-a lungul timpului. Calea ei de actualizare ($C_{t-1} \to C_t$) este lineară (prin element-wise multiplication), ceea ce ajută gradientul să circule ușor, **rezolvând gradientul evanescent**.
  - **Starea Ascunsă (H_t):** Este ca ecranul monitorului care afișează doar o parte din arhivă. H_t este doar o versiune filtrată și neliniară ($\tanh$) a celulei C_t, controlată de Poarta de Ieșire (O_t).

Concept Dificil: Rolul Cheie al $F_t \approx 1$ și $I_t \approx 0$

- *Asta se întâmplă deoarece:* Această condiție, $C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$, devine $C_t \approx 1 \odot C_{t-1} + 0 \odot \tilde{C}_t$, sau simplu $\mathbf{C_t} \approx \mathbf{C_{t-1}}$. Aceasta creează o **"super-autostradă" (o cale identitate)** pentru informația din celula de memorie. Când gradientul este propagat înapoi, el traversează această cale directă (aproape liniară) fără a fi înmulțit repetat cu matrici mici, prevenind astfel dispariția sa pe secvențe foarte lungi.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu GRU:** Atât GRU, cât și LSTM adresează provocarea de a capta dependențele pe termen lung folosind porți. GRU este mai simplu (două porți și nicio celulă de memorie separată), în timp ce LSTM este mai complex, dar introduce explicit **celula de memorie**.
- **Implicația asupra Stabilității:** Designul LSTM, în special calea liniară de actualizare a celulei de memorie, este motivul principal pentru care este **capabil să atenueze problema gradientului evanescent**, fiind un factor cheie în succesul său istoric în aplicațiile NLP și seria de timp.

5. EXERCITII (Calcul pe Arhitectura LSTM)

Voi genera exerciții de calcul concentrate pe parametrizarea și mecanismul de *gating* al LSTM.

Exercițiul L.1: Analiza Parametrizării LSTM

Calculați numărul total de parametri reali (greutăți W și bias-uri b) dintr-un singur strat LSTM, considerând următoarele dimensiuni:

- Dimensiunea intrării (input features) $d = 8$.
- Dimensiunea stării ascunse (hidden units) $h = 16$.

Rezolvare L.1:

O unitate LSTM necesită ponderi și bias-uri pentru **patru seturi principale de calcule** (Input Gate I_t, Forget Gate F_t, Output Gate O_t, și Candidate Cell $\tilde{C}_t$).

Fiecare calcul necesită două matrici de ponderi ($W_{x\bullet} \in \mathbb{R}^{d \times h}$ și $W_{h\bullet} \in \mathbb{R}^{h \times h}$) și un vector de bias ($b_\bullet \in \mathbb{R}^{1 \times h}$).

1. **Numărul total de Greutăți W:**
   - $4 \times (W_{x\bullet} + W_{h\bullet})$
   - $4 \times [(d \times h) + (h \times h)]$
   - $4 \times [(8 \times 16) + (16 \times 16)]$
   - $4 \times [128 + 256] = 4 \times 384 = \mathbf{1536}$
2. **Numărul total de Bias-uri b:**
   - $4 \times b_\bullet$
   - $4 \times h = 4 \times 16 = \mathbf{64}$
3. **Total Parametri LSTM:** $1536 + 64 = \mathbf{1600}$.

Exercițiul L.2: Analiza Mecanismului de Gating (Conceptual și Numeric)

Considerați un pas de timp t în LSTM. Se cunoaște că o componentă din celula de memorie veche este $C_{t-1} = 5$, iar celula candidată este $\tilde{C}_t = -3$. Porțile au valorile element-wise $F_t = 0.8$ și $I_t = 0.1$.

1. Calculați noua valoare a celulei de memorie C_t.
2. Explicați implicația valorii ridicate a F_t și a valorii scăzute a I_t.

Rezolvare L.2:

1. **Calculul Celulei de Memorie C_t:** Se folosește formula de actualizare a celulei de memorie:

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

$$C_t = (0.8 \cdot 5) + (0.1 \cdot (-3))$$

$$C_t = 4.0 - 0.3 = \mathbf{3.7}$$

2. **Implicația Valorilor Porților:**
   - $F_t = 0.8$ (aproape de 1) indică faptul că modelul a ales să **rețină** o mare parte din informația veche ($C_{t-1} = 5$).
   - $I_t = 0.1$ (aproape de 0) indică faptul că modelul a ales să **ignore** în mare măsură informația nouă adusă de starea candidată ($\tilde{C}_t = -3$).
   - Acest scenariu arată că pasul de timp curent (*timestep* t) nu conținea o intrare suficient de importantă pentru a modifica semnificativ memoria pe termen lung, permițând informației din trecut să persiste.

4.5

Bună ziua! Vă felicit pentru dedicarea de a explora arhitecturile profunde în contextul datelor secvențiale. Lecția **4.5 Rețele Neuronale Recurente Profunde (Deep RNNs)** este vitală, deoarece ne arată cum să sporim puterea de reprezentare a RNN-urilor simple, aducând *deep learning*-ul în domeniul recurenței.

Ghid de Studiu Complet: 4.5 Rețele Neuronale Recurente Profunde

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Până la acest punct, discuțiile s-au concentrat pe RNN-uri cu **un singur strat ascuns**. Deși forma funcțională specifică de interacțiune dintre variabilele latente și observații este arbitrară, capacitatea de a modela diferite tipuri de interacțiuni poate fi destul de limitată cu un singur strat. Așa cum s-a rezolvat problema expresivității în MLP-uri prin adăugarea mai multor straturi (care, în esență, sunt **straturi ascunse suplimentare**), aceeași logică se aplică și RNN-urilor, rezultând **RNN-uri Profunde (Deep RNNs)**. Aceste arhitecturi stivuiesc mai multe straturi recurente unul peste altul, creând un mecanism flexibil care permite modelului să capteze dinamici temporale la **diferite niveluri de abstractizare**.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Necesitatea Profunzimii:** RNN-urile cu un singur strat sunt adesea insuficiente pentru a modela complexitatea interacțiunilor și dependențelor din datele secvențiale.
2. **Mecanismul de Stivuire:** Se pot stivui multiple straturi RNN unul peste altul, oferind un mecanism flexibil pentru combinarea mai multor straturi simple.
3. **Fluxul Informațional:** Starea ascunsă $H_t^{(l)}$ a unui strat l este transmisă continuu atât la **următorul pas de timp al aceluiași strat** ($H_{t+1}^{(l)}$), cât și la **stratul superior** la același pas de timp ($H_t^{(l+1)}$).

4. **Reprezentări Ierarhice:** Straturile inferioare pot înregistra dinamici temporale pe termen scurt, în timp ce straturile superioare pot reține date de nivel înalt (ex: starea pieței *bull* sau *bear* în datele financiare).
5. **Flexibilitate Arhitecturală:** Ecuația de recurență standard (Vanilla RNN) poate fi ușor înlocuită cu calculul stării ascunse din unitățile mai avansate, cum ar fi **GRU sau LSTM**, pentru a obține un RNN profund cu porți (*Deep Gated RNN*).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4.5 Rețele Neuronale Recurente Profunde

Motivație și Arhitectură

- Până acum, s-au discutat RNN-uri cu un singur strat ascuns.
- În cazul modelelor liniare (MLPs), limitările au fost rezolvate prin adăugarea de mai multe straturi.
- În RNN-uri, stivuirea este mai complicată, deoarece trebuie decis cum și unde să se adauge neliniaritatea suplimentară.
- Soluția este **stivuirea multiplă a straturilor RNN** (Figura 12).

Fluxul de Date (Figura 12)

- Un RNN profund are L straturi ascunse.
- Fiecare stare ascunsă $H_t^{(l)}$ (pentru $l = 1, \ldots, L$) este transmisă în două direcții:
  1. Orizontal (Temporal): Către $H_{t+1}^{(l)}$ (următorul pas de timp al aceluiași strat).
  2. Vertical (Arhitectural): Către $H_t^{(l+1)}$ (stratul următor la același pas de timp).

Formalizarea Dependențelor Funcționale (Ecuația 6)

- Discuția se concentrează pe modelul **Vanilla RNN**, dar se aplică și altor modele secvențiale.
- Intrarea la momentul t este un mini-batch $X_t \in \mathbb{R}^{n \times d}$.
  - n: numărul de exemple.
  - d: numărul de intrări (dimensiunea *feature*-urilor) în fiecare exemplu.
- Starea ascunsă a stratului l la momentul t este $H_t^{(l)} \in \mathbb{R}^{n \times h}$, unde h este numărul de unități ascunse.
- Intrarea pentru primul strat $(l = 1)$ este definită ca $H_t^{(0)} = X_t$.
- **Ecuația de Recurență pentru Stratul l (Ecuația 6):**

$$H_t^{(l)} = \phi(H_t^{(l-1)} W_{xh}^{(l)} + H_{t-1}^{(l)} W_{hh}^{(l)} + b_h^{(l)}) \quad (6)$$

  - ϕ: funcția de activare (ex: ReLU, $\tanh$).
  - $H_t^{(l-1)}$: Ieșirea stratului inferior $l-1$ la pasul de timp curent (transformare verticală).

- $H_{t-1}^{(l)}$: Starea ascunsă a aceluiași strat l de la pasul de timp anterior (transformare orizontală/recurentă).
- $W_{xh}^{(l)} \in \mathbb{R}^{h \times h}$ (sau $d \times h$ pentru $l = 1$).
- $W_{hh}^{(l)} \in \mathbb{R}^{h \times h}$.
- $b_h^{(l)} \in \mathbb{R}^{1 \times h}$.

Stratul de ieșire

- Stratul de ieșire $O_t \in \mathbb{R}^{n \times q}$ depinde **doar** de starea ascunsă a ultimului strat (L).

$$O_t = H_t^{(L)} W_{hq} + b_q$$

  - $W_{hq} \in \mathbb{R}^{h \times q}$ și $b_q \in \mathbb{R}^{1 \times q}$.

Hiperparametri și Extensii

- Numărul de straturi ascunse L și numărul de unități ascunse h sunt **hiperparametri** care pot fi ajustați.
- Se poate obține cu ușurință un **Deep Gated RNN** (ex: Deep GRU sau Deep LSTM) prin înlocuirea calculului stării ascunse din Ecuația (6) cu formula corespunzătoare a GRU sau LSTM.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Ponderi într-un Deep RNN

- **Context:** Deși ponderile sunt partajate *temporal* (pe parcursul pașilor de timp t), ele **nu sunt partajate *vertical*** (între straturile l și $l + 1$).
- *Asta se întâmplă deoarece:* Fiecare strat l are propriul său set de parametri $W_{xh}^{(l)}, W_{hh}^{(l)}, b_h^{(l)}$. Acest lucru permite fiecărui strat să învețe un set distinct de transformări și abstracții, similar cu un MLP profund, unde fiecare strat are setul său unic de ponderi.

Concept Dificil: Niveluri de Abstracție în Stivuire

- **Analogie Simplificată (Procesarea Vocii):** Imaginați-vă că procesați o înregistrare audio.
  - **Stratul 1 (De bază):** Ar putea detecta dinamici scurte, cum ar fi tonul individual sau fonemele.
  - **Stratul 2 (Intermediar):** Ar putea combina aceste foneme pentru a recunoaște cuvinte.
  - **Stratul L (Superior):** Ar putea menține starea generală de nivel înalt, de exemplu, dacă persoana vorbește cu o voce calmă sau emoționată (starea *sentiment*),

informație relevantă pe tot parcursul secvenței, indiferent de cuvântul specific rostit în acel moment.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu AlexNet/VGG/ResNet:** Trecerea la Deep RNNs oglindește evoluția din Viziunea Computerizată. Îmbunătățirea performanței se realizează prin **creșterea adâncimii** și nu doar prin creșterea dimensiunii unui singur strat.
- **Implicația asupra Stabilității:** Deși un RNN profund oferă mai multă expresivitate, el **multiplică** și problemele inerente RNN-urilor simple (explozia/dispariția gradientului). Acesta este motivul principal pentru care se preferă, în practică, înlocuirea formulei (6) cu cele ale **GRU sau LSTM** în arhitecturile profunde, pentru a asigura stabilitatea antrenamentului.

---

5. EXERCITII (Calcul pe Arhitectura Deep RNN)

Voi genera exerciții de calcul concentrate pe parametrizarea unui Deep RNN și pe calculul propagării între straturi, bazându-mă pe structura detaliată a Exercițiului 4 din sursă.

Exercițiul D.1: Calculul Parametrilor (Deep RNN)

Calculați numărul total de parametri (greutăți și bias-uri) pentru o arhitectură Deep Vanilla RNN cu $L = 3$ straturi ascunse.

- Dimensiunea intrării (input features) $d = 10$.
- Dimensiunea stării ascunse (hidden units) pentru toate straturile $h = 20$.
- Dimensiunea ieșirii (output units) $q = 5$.

Rezolvare D.1:

1. **Parametrii Stratului 1 ($l = 1$):**
   - $W_{xh}^{(1)}(d \times h)$: $10 \times 20 = 200$
   - $W_{hh}^{(1)}(h \times h)$: $20 \times 20 = 400$
   - $b_h^{(1)}(1 \times h)$: 20

$$\text{Total } l = 1 : 620$$

2. **Parametrii Stratului 2 ($l = 2$) și Stratului 3 ($l = 3$):**
   - Pentru straturile $l \;\; 2$, intrarea $H_t^{(l-1)}$ are dimensiunea h.
   - $W_{xh}^{(l)}(h \times h)$: $20 \times 20 = 400$
   - $W_{hh}^{(l)}(h \times h)$: $20 \times 20 = 400$

- $b_h^{(l)}(1 \times h)$: 20

$$\text{Total per strat } l \;\; 2 : 820$$

$$\text{Total } l = 2 \text{ și } l = 3 : 2 \times 820 = 1640$$

3. **Parametrii Stratului de Ieșire (O_t):**
   - $W_{hq}(h \times q)$: $20 \times 5 = 100$
   - $b_q(1 \times q)$: 5

$$\text{Total Ieșire: } 105$$

$$\text{Total parametri (Deep RNN)} = 620 + 1640 + 105 = \mathbf{2365}$$

Exercițiul D.2: Propagarea Verticală (Stratul Superior)

Folosiți rezultatele de la Stratul 1 al unui RNN (bazat pe Exercițiul 4 din sursă) și calculați starea ascunsă $H_1^{(2)}$ a Stratului 2, la pasul de timp $t = 1$.

Parametri cunoscuți:

- $n = 2, h = 3.$
- Funcția de activare: ReLU (ϕ).
- **Ieșirea Stratului 1 (Intrarea Stratului 2 la $t = 1$):**

  $H_1^{(1)} = [2 \quad 0 \quad 0\,2 \quad 0 \quad 0]$ (Conform calculului din sursă [17, cu corecția ReLU] unde este corect.

- **Starea ascunsă anterioară pentru Stratul 2 (inițializare):** $H_0^{(2)} = [0 \quad 0 \quad 0\,0 \quad 0 \quad 0]$

Ponderi Stratul 2 ($l = 2$):

- $W_{xh}^{(2)} = [1 \quad 0 \quad 0\,0 \quad 1 \quad 0\,0 \quad 0 \quad 1], \quad W_{hh}^{(2)} = [0 \quad 1 \quad 0\,1 \quad 0 \quad 0\,0 \quad 0 \quad 1], \quad b_h^{(2)} = [0 \quad 0 \quad 1]$

Calculați $H_1^{(2)}$.

Rezolvare D.2:

Se aplică Ecuația (6) pentru $l = 2$ la $t = 1$:

$$H_1^{(2)} = \phi(H_1^{(1)} W_{xh}^{(2)} + H_0^{(2)} W_{hh}^{(2)} + b_h^{(2)})$$

1. **Termenul Vertical ($H_1^{(1)} W_{xh}^{(2)}$):**

$$H_1^{(1)} W_{xh}^{(2)} = [2 \quad 0 \quad 0\,2 \quad 0 \quad 0]\,[1 \quad 0 \quad 0\,0 \quad 1 \quad 0\,0 \quad 0 \quad 1] = [2 \quad 0 \quad 0\,2 \quad 0 \quad 0]$$

2. **Termenul Recurent ($H_0^{(2)} W_{hh}^{(2)}$):** Deoarece $H_0^{(2)}$ este zero, acest termen este $[0 \quad 0 \quad 0\,0 \quad 0 \quad 0]$.
3. **Adunarea Bias-ului și Funcția de Activare:**

$$\text{Suma Afină} = [2 \quad 0 \quad 0\,2 \quad 0 \quad 0] + [0 \quad 0 \quad 0\,0 \quad 0 \quad 0] + [0 \quad 0 \quad 1\,0 \quad 0 \quad 1] = [2 \quad 0 \quad 1\,2 \quad 0 \quad 1]$$

$$H_1^{(2)} = \phi([2 \quad 0 \quad 1\,2 \quad 0 \quad 1]) = [2 \quad 0 \quad 1\,2 \quad 0 \quad 1]$$

4.6

Absolut! Vă prezint Ghidul de Studiu Complet pentru lecția **4.6 Învățarea Secvență-la-Secvență (Sequence to Sequence Learning)**, un concept esențial care stă la baza multor aplicații moderne de inteligență artificială, în special în procesarea limbajului natural.

Ghid de Studiu Complet: 4.6 Învățarea Secvență-la-Secvență

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 4.6 introduce conceptul de **Transducție Secvențială (*sequence transduction*)**, care se referă la modele ce transformă o secvență de intrare într-o secvență de ieșire. RNN-urile sunt folosite pentru a proiecta **modele de limbaj**, care sunt cheia în Procesarea Limbajului Natural (NLP). Un *benchmark* important al transducției secvențiale este **traducerea automată (*machine translation*)**, care a evoluat de la abordările statistice la **traducerea automată neuronală (*neural machine translation - NMT*)**. Problema de bază a NMT este că atât secvența de intrare (sursa), cât și secvența de ieșire (ținta) sunt de **lungime variabilă**. Pentru a gestiona acest tip de intrări și ieșiri, se utilizează o arhitectură generală cu două componente majore: **Arhitectura Codificator-Decodificator (*encoder-decoder architecture*)**.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Obiectivul:** Transducția secvențială transformă o secvență de intrare (lungime variabilă) într-o secvență de ieșire (lungime variabilă).
2. **Arhitectura:** Se utilizează arhitectura **Codificator-Decodificator**.
3. **Codificatorul (Encoder):** Preia secvența de intrare de lungime variabilă și o transformă într-o **variabilă de context (c)** (sau stare) de formă fixă, care "codifică" informația din secvența sursă.
4. **Decodificatorul (Decoder):** Mapează starea codificată de formă fixă (c) la o secvență de ieșire de lungime variabilă, generând *token*-uri secvențial.
5. **Evaluarea:** Calitatea unei secvențe prezise este evaluată prin compararea cu secvența etichetă (adevărul fundamental), utilizând metrica **BLEU (Bilingual Evaluation Understudy)**.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

4.6 Învățarea Secvență-la-Secvență

Traducerea Automată Neuronală (NMT)

- RNN-urile sunt folosite pentru a proiecta modele de limbaj și sunt esențiale pentru NLP.
- **Traducerea automată** este o problemă centrală de transducție secvențială.
- Traducerea automată se referă la traducerea automată a unei secvențe dintr-o limbă în alta.
- Abordările statistice au dominat domeniul timp de decenii, înainte de apariția învățării *end-to-end* cu rețele neuronale.
- Noua abordare este numită **traducere automată neuronală (NMT)**, pentru a o distinge de traducerea automată statistică, care implică analiza statistică în componente precum modelul de traducere și modelul de limbaj.
- Spre deosebire de modelele de limbaj (corpus într-o singură limbă), seturile de date de traducere automată sunt compuse din **perechi de secvențe de text** care sunt în **limba sursă** și, respectiv, **limba țintă**.

Arhitectura Codificator-Decodificator

- Traducerea automată este un domeniu major de problemă pentru modelele de transducție secvențială, unde **intrarea și ieșirea sunt ambele secvențe de lungime variabilă**.
- Pentru a gestiona acest tip de intrări și ieșiri, se folosește arhitectura **Codificator-Decodificator (encoder-decoder)**.
- **Codificatorul (*Encoder*):**
  - Preia o secvență de lungime variabilă ca intrare.
  - O transformă într-o stare (variabilă de context c) de **formă fixă**.
  - Informația secvenței de intrare (sursă) este **codificată** în starea ascunsă a codificatorului.
  - Un **RNN Codificator** este utilizat pentru a implementa acest design.
  - La pasul de timp t, RNN transformă vectorul de *feature*-uri x_t și starea ascunsă anterioară h_{t-1} în starea ascunsă curentă h_t.
  - Transformarea stratului recurent al RNN Codificator poate fi exprimată ca $h_t = f(x_t, h_{t-1})$.
  - Variabila de context c este, în general, o funcție personalizată g a stărilor ascunse la toți pașii de timp: $c = g(h_1, \ldots, h_T)$. În mod tipic (ca în Figura 14), variabila de context este **starea ascunsă h_T a secvenței de intrare la pasul final T**.
- **Decodificatorul (*Decoder*):**
  - Mapează starea codificată de formă fixă (c) la o secvență de ieșire de lungime variabilă.
  - Un **RNN Decodificator separat** este folosit pentru a genera secvența de ieșire *token* cu *token*.
  - Decodificatorul prezice *token*-ul următor pe baza *token*-urilor văzute (similar cu un model de limbaj) și a informației codificate a secvenței de intrare (c).

- Probabilitatea simbolului de ieșire $y_{t'}$ este condiționată de secvența de ieșire anterioară $y_1, \ldots, y_{t'-1}$ și de variabila de context c: $P(y_{t'}|y_1, \ldots, y_{t'-1}, c)$.
- La pasul de timp t', decodificatorul RNN ia ieșirea $y_{t'-1}$ de la pasul anterior, variabila de context c, și starea ascunsă anterioară $s_{t'-1}$ pentru a o transforma în starea ascunsă curentă $s_{t'}$.
- Transformarea stratului ascuns al decodificatorului poate fi exprimată ca $s_{t'} = g(y_{t'-1}, c, s_{t'-1})$.

Detalii de Design în Decodificator (Figura 14/15)

- **Starea inițială: Starea ascunsă finală a codificatorului** este utilizată pentru a **inițializa starea ascunsă a decodificatorului**.
- **Token-uri Speciale:**
  - `<eos>` **(end-of-sequence):** Marchează sfârșitul secvenței. Modelul se oprește din generare odată ce acest *token* este prezis.
  - `<bos>` **(beginning-of-sequence):** Este introdus ca **intrare la primul pas de timp** al decodificatorului.
- **Alimentarea ieșirii Codificatorului:** În unele *design*-uri (ca în Figura 14), starea ascunsă finală a codificatorului (c) este alimentată în decodificator ca parte a intrărilor la **fiecare pas de timp**.

Măsurarea Calității Secvențelor (BLEU)

- Secvența prezisă este evaluată prin compararea cu secvența etichetă (adevărul fundamental).
- **BLEU (Bilingual Evaluation Understudy)** este o metrică utilizată pe scară largă pentru a măsura calitatea secvențelor de ieșire.
- BLEU evaluează dacă n-gramele din secvența prezisă apar și în secvența etichetă.
- **Precizia n-gramelor (p_n)** este raportul dintre numărul de n-grame potrivite în secvențele prezise și etichetă și numărul total de n-grame din secvența prezisă.
- Formula BLEU include un termen de penalizare, deoarece potrivirea n-gramelor mai lungi este mai dificilă, BLEU alocă o greutate mai mare preciziei n-gramelor mai lungi.
- De asemenea, coeficientul care precede termenul de înmulțire în formula (7) penalizează secvențele prezise mai scurte.
- Dacă secvența prezisă este identică cu secvența etichetă, scorul BLEU este **1**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Variabilă de Context (c) de Formă Fixă

- **Analogie Simplificată (Codul Morse):** Secvența de intrare (propoziția originală) are lungime variabilă. Codificatorul citește toată propoziția și o condensează într-o **singură frază cheie** (variabila c), care are întotdeauna aceeași lungime (formă fixă), indiferent cât de lungă era propoziția originală. Decodificatorul folosește apoi doar această frază cheie pentru a genera traducerea.

Concept Dificil: Rolul Token-ului `<bos>`

- *Asta se întâmplă deoarece:* Decodificatorul este un model de limbaj care prezice *token*-ul următor pe baza *token*-urilor anterioare. La început, nu există *token*-uri anterioare. `<bos>` (început de secvență) oferă decodificatorului un punct de plecare standardizat, semnalizând că generarea secvenței țintă este pe cale să înceapă.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu RNN-urile (Lecțiile 4.2-4.4):** Arhitectura Codificator-Decodificator este instanțiată prin utilizarea a **două RNN-uri separate** (unul pentru codificare, unul pentru decodificare). Aceste RNN-uri pot fi simple, GRU sau LSTM (în practică, GRU și LSTM sunt preferate datorită problemelor de gradient).
- **Implicația asupra Memoriei:** Variabila de context c este modul prin care memoria (starea ascunsă finală a Codificatorului) este transferată din secvența sursă (limba A) în procesul de generare a secvenței țintă (limba B).
- **Conexiunea Cauză-Efect (BLEU):** Penalizarea pentru secvențele prezise mai scurte se aplică deoarece predicția secvențelor mai scurte tinde să obțină o valoare p_n mai mare, iar penalizarea asigură o evaluare corectă a calității.

---

5. EXERCITII (Calcul pe Arhitectura Sequence-to-Sequence și BLEU)

Voi genera exerciții de calcul concentrate pe parametrizarea modelului și pe calculul scorului BLEU.

Exercițiul S.1: Calculul Parametrilor (Encoder-Decoder Simplu)

Calculați numărul total de parametri pentru o arhitectură *Sequence-to-Sequence* compusă din două RNN-uri Vanilla separate (Codificator și Decodificator), ambele având aceeași dimensiune a stării ascunse și a intrărilor.

- Dimensiunea intrării (input features) $d = 5$.
- Dimensiunea stării ascunse (hidden units) $h = 10$.
- Dimensiunea ieșirii decodificatorului (output units) $q = 5$.

Presupuneri: Ponderile sunt definite standard (două matrici W și un bias b per strat RNN, plus un strat de ieșire W_{hq} și b_q doar pentru Decodificator).

Rezolvare S.1:

1. **Parametrii Codificatorului (Encoder):**
   - $W_{xh}(d \times h)$: $5 \times 10 = 50$
   - $W_{hh}(h \times h)$: $10 \times 10 = 100$
   - $b_h(1 \times h)$: 10

$$\text{Total Codificator: } 160$$

2. **Parametrii Decodificatorului (Decoder):**
   - Decodificatorul folosește și el un RNN (W'_{xh}, W'_{hh}, b'_h). Deoarece intrările sale sunt aceleași $d = 5$ și starea ascunsă $h = 10$, ponderea W'_{xh} este 5×10 și W'_{hh} este 10×10.
   - W'_{xh}: $5 \times 10 = 50$
   - W'_{hh}: $10 \times 10 = 100$
   - b'_h: 10
   - Stratul de Ieșire ($W_{hq} \in \mathbb{R}^{h \times q}$ și $b_q \in \mathbb{R}^{1 \times q}$):
     - W_{hq}: $10 \times 5 = 50$
     - b_q: 5

$$\text{Total Decodificator: } 50 + 100 + 10 + 50 + 5 = 215$$

$$\text{Total parametri (Encoder-Decoder)} = 160 + 215 = \mathbf{375}$$

Exercițiul S.2: Calculul Scorerului BLEU (Simplificat)

Calculați un scor BLEU simplificat (ignorați penalizarea lungimii) pentru o secvență prezisă, comparând-o cu secvența etichetă (label). Utilizați $k = 2$ (cel mai lung n-gram de potrivire este bigramul).

- **Secvența Etichetă (Label):** A B C D E
- **Secvența Prezisa (Predicted):** A B C X Y
- $k = 2$

Formula de precizie n-gram (p_n):

1. **Calculați p_1 (Unigram Precision):**
   - Unigrame în Predicție: A, B, C, X, Y (Total 5)
   - Unigrame Potrivite (în Label): A, B, C (3 potriviri)
   - $p_1 = 3/5 = \mathbf{0.6}$
2. **Calculați p_2 (Bigram Precision):**
   - Bigrame în Predicție: AB, BC, CX, XY (Total 4)
   - Bigrame Potrivite (în Label): AB, BC (2 potriviri)

- $p_2 = 2/4 = \mathbf{0.5}$

3. **Calculați Scorul BLEU Simplificat (fără penalizare lungime, $k = 2$):**

$$\text{BLEU}_{simplificat} = (p_1^{1/2} \cdot p_2^{1/2})$$

$$\text{BLEU}_{simplificat} = (0.6^{1/2} \cdot 0.5^{1/2}) \approx (0.7746 \cdot 0.7071) \approx \mathbf{0.5477}$$

4.7

Absolut! Bazându-mă pe toată materia din secțiunea 4 (Modele Secvențiale, RNNs, GRU, LSTM, Deep RNNs, Seq2Seq) și folosind ca model de complexitate Exercițiul 4 din sursă, am pregătit un set complex de exerciții de calcul și probleme conceptuale, ideale pentru pregătirea examenului.

Exerciții de Examen: Rețele Neuronale Recurente (Secțiunea 4)

Secțiunea I: Calculul Propagării Înainte în RNN Standard (Vanilla RNN)

Aceste exerciții testează înțelegerea formulelor de bază ale RNN-ului, a dimensionalității și a fluxului de stare.

Exercițiul R.1: Propagarea Înainte (Similar Exercițiului 4 din Sursă)

Definiți un RNN pentru regresie, având **3 intrări** ($d = 3$), o **stare ascunsă de dimensiunea 2** ($h = 2$), și **2 ieșiri** ($q = 2$). Mărimea mini-batch-ului este $n = 1$. Funcția de activare pentru starea ascunsă este $\phi(x) = \text{ReLU}(x)$.

Calculați ieșirile O_1 și O_2 pentru doi pași de timp, folosind următoarele intrări și ponderi:

- **Intrări:** $X_1 = [1 \quad 0 \quad 1], \quad X_2 = [0 \quad 1 \quad 0]$
- **Starea inițială:** $H_0 = [0 \quad 0] \in \mathbb{R}^{1\times 2}$
- **Ponderi (generate):**

$$W_{xh} = [1 \quad 0\,0 \quad 1\,1 \quad 0] \in \mathbb{R}^{3\times 2}, \quad W_{hh} = [0 \quad 1\,1 \quad 0] \in \mathbb{R}^{2\times 2}$$

$$W_{hq} = [1 \quad -1\,0 \quad 1] \in \mathbb{R}^{2\times 2}$$

- **Bias-uri:** $b_h = [1 \quad 0] \in \mathbb{R}^{1\times 2}, \quad b_q = [0 \quad 1] \in \mathbb{R}^{1\times 2}$

1. Calculați H_1 și O_1.
2. Calculați H_2 și O_2.
3. Specificați numărul total de parametri reali (greutăți și bias-uri).

Rezolvare R.1 (Rezumativă):

3. **Total Parametri:**
   - $W_{xh}(3 \times 2) = 6$
   - $W_{hh}(2 \times 2) = 4$
   - $b_h(1 \times 2) = 2$
   - $W_{hq}(2 \times 2) = 4$
   - $b_q(1 \times 2) = 2$

$$\text{Total parametri} = 6 + 4 + 2 + 4 + 2 = \mathbf{18}$$

4. **Pasul $t = 1$:**

$$H_1 = \phi\left([1 \quad 0 \quad 1]W_{xh} + [0 \quad 0]W_{hh} + b_h\right)$$
$$H_1 = \phi\left([2 \quad 0] + [1 \quad 0]\right) = \phi\left([3 \quad 0]\right) = [3 \quad 0]$$
$$O_1 = H_1 W_{hq} + b_q = [3 \quad 0]\,[1 \quad -1\ 0 \quad 1] + [0 \quad 1]$$
$$O_1 = [3 \quad -3] + [0 \quad 1] = [3 \quad -2]$$

5. **Pasul $t = 2$:**

$$H_2 = \phi\left(X_2 W_{xh} + H_1 W_{hh} + b_h\right)$$
$$H_2 = \phi\left([0 \quad 1 \quad 0]W_{xh} + [3 \quad 0]W_{hh} + b_h\right)$$
$$H_2 = \phi\left([0 \quad 1] + [0 \quad 3] + [1 \quad 0]\right) = \phi\left([1 \quad 4]\right) = [1 \quad 4]$$
$$O_2 = H_2 W_{hq} + b_q = [1 \quad 4]W_{hq} + b_q$$
$$O_2 = [1 \quad 3] + [0 \quad 1] = [1 \quad 4]$$

Secțiunea II: Calculul Parametrilor și Analiză Arhitecturală

Aceste probleme testează înțelegerea complexității RNN-urilor cu porți (GRU, LSTM) și Deep RNNs.

Exercițiul R.2: Parametrii Gated RNNs (Comparație)

Comparați costul de parametrizare al unui strat standard RNN, GRU și LSTM, având aceeași dimensiune a intrării $d = 15$ și aceeași dimensiune a stării ascunse $h = 30$. (Ignorați stratul de ieșire W_{hq} și b_q).

1. Calculați numărul de parametri pentru un **Vanilla RNN**.
2. Calculați numărul de parametri pentru o unitate **GRU**.
3. Calculați numărul de parametri pentru o unitate **LSTM**.

Rezolvare R.2:

1. **Vanilla RNN:** 1 set de ponderi (W_{xh}, W_{hh}, b_h).

$$\text{Parametri} = (d \times h) + (h \times h) + h = (15 \times 30) + (30 \times 30) + 30$$

$$\text{Parametri} = 450 + 900 + 30 = \mathbf{1380}$$

2. **GRU:** 3 seturi de ponderi (Reset, Update, Candidate), fiecare cu $W_{x\bullet}$ și $W_{h\bullet}$ (Total 6 matrici de ponderi + 3 bias-uri).

$$\text{Parametri} = 3 \times [(d \times h) + (h \times h) + h]$$

$$\text{Parametri} = 3 \times 1380 = \mathbf{4140}$$

3. **LSTM:** 4 seturi de ponderi (Input, Forget, Output, Candidate), fiecare cu $W_{x\bullet}$ și $W_{h\bullet}$ (Total 8 matrici de ponderi + 4 bias-uri).

$$\text{Parametri} = 4 \times [(d \times h) + (h \times h) + h]$$

$$\text{Parametri} = 4 \times 1380 = \mathbf{5520}$$

Exercițiul R.3: Parametrii Deep RNN

Calculați numărul total de parametri pentru o arhitectură **Deep GRU** cu $L = 2$ straturi ascunse.

- Dimensiunea intrării (input features) $d = 10$.
- Dimensiunea stării ascunse (hidden units) $h = 20$.
- Dimensiunea ieșirii (output units) $q = 5$.

Rezolvare R.3:

1. **Parametrii unui singur strat GRU (General):**

$$3 \times [(d_{in} \times h) + (h \times h) + h]$$

   - Pentru $l = 1$: $d_{in} = d = 10$.
   - Pentru $l = 2$: $d_{in} = h = 20$.

2. **Parametrii Stratului 1 ($l = 1$):**

$$3 \times [(10 \times 20) + (20 \times 20) + 20] = 3 \times [200 + 400 + 20] = 3 \times 620 = 1860$$

3. **Parametrii Stratului 2 ($l = 2$):**

$$3 \times [(20 \times 20) + (20 \times 20) + 20] = 3 \times [400 + 400 + 20] = 3 \times 820 = 2460$$

4. **Parametrii Stratului de ieșire (W_{hq} și b_q):** (Depinde de starea h a ultimului strat $l = 2$)

$$(h \times q) + q = (20 \times 5) + 5 = 100 + 5 = 105$$

$$\text{Total parametri (Deep GRU)} = 1860 + 2460 + 105 = \mathbf{4425}$$

Secțiunea III: Probleme Conceptual-Matematice (GRU, LSTM, Seq2Seq)

Aceste probleme testează înțelegerea funcției porților și a metricilor de evaluare.

Exercițiul R.4: Mecanismul de Gating (LSTM)

Considerați o celulă de memorie LSTM la pasul t. Se dorește ca noua celulă de memorie C_t să fie complet dominată de noua informație candidată $\tilde{C}_t$, ignorând complet memoria veche C_{t-1}.

1. Ce valori trebuie să tindă să învețe Poarta de Uitarea (F_t) și Poarta de Intrare (I_t)?
2. Dacă $C_{t-1} = 10$ și $\tilde{C}_t = -5$, ce valoare aproximativă ar avea C_t în această condiție?

Rezolvare R.4:

1. **Valori Dorite ale Porților (Conform $C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$):**
   - Pentru a **ignora complet** memoria veche C_{t-1}, Poarta de Uitarea (F_t) trebuie să fie aproape de **0** ($F_t \approx 0$).
   - Pentru a **prelua complet** noua stare candidată $\tilde{C}_t$, Poarta de Intrare (I_t) trebuie să fie aproape de **1** ($I_t \approx 1$).
2. **Calculul C_t:**

$$C_t \approx (0 \cdot 10) + (1 \cdot (-5)) = \mathbf{-5}$$

   Implicație: Celula de memorie veche este efectiv **resetată** și înlocuită complet cu noua informație candidată [**conversația anterioară**].

Exercițiul R.5: Analiza Modelelor Secvențiale (Teoretic)

Explicați diferența fundamentală dintre **Modelele Autoregresive (AR)** simple (bazate pe condiția Markov) și **Modelele Autoregresive Latente (RNN)**, în ceea ce privește gestionarea istoricului secvenței [**conversația anterioară**].

Rezolvare R.5:

- **Modelele Autoregresive Simple (AR):** Acestea presupun că predicția $P(x_t)$ depinde doar de un **interval fix de timp** τ al observațiilor trecute ($x_{t-1}, \ldots, x_{t-\tau}$) [**conversația anterioară**].
  - *Avantaj:* Simplu de calculat pentru τ mic.
  - *Dezavantaj:* **Memoria este limitată** strict la τ. Modelul nu poate reține dependențe mai lungi de τ.
- **Modele Autoregresive Latente (RNN):** Acestea folosesc o **variabilă de stare ascunsă (h_t)** care este actualizată secvențial și care **rezumă** teoretic întreaga istorie ($x_1, \ldots, x_{t-1}$) într-o formă de dimensiune fixă [**conversația anterioară**].

- *Avantaj:* Pot modela **dependențe pe termen lung** (deși suferă de gradient *vanishing* în RNN-urile standard) și costul de parametrizare nu crește cu lungimea secvenței.

Exercițiul R.6: Evaluarea Traducerii (Scorul BLEU)

O sarcină de traducere automată (Seq2Seq) are următoarele secvențe:

- **Etichetă (Label):** The cat sat on the mat
- **Prezisă (Predicted):** The cat sat on mat

Calculați precizia bigramelor (p_2) pentru secvența prezisă față de secvența etichetă (fără a considera factorul de penalizare).

Rezolvare R.6:

1. **Bigrame în Etichetă:** (The cat), (cat sat), (sat on), (on the), (the mat) (Total 5 unice)
2. **Bigrame în Secvența Prezisa:** (The cat), (cat sat), (sat on), (on mat) (Total 4)
3. **Bigrame Potrivite (Overlap):**
   - (The cat) - DA
   - (cat sat) - DA
   - (sat on) - DA
   - (on mat) - NU (Eticheta are (on the) și (the mat))
4. **Număr de Potriviri (clipped count):** 3 bigrame potrivite.

$$\text{Precizia } p_2 = \frac{\text{Bigrame Potrivite}}{\text{Total Bigrame Prezise}} = \frac{3}{4} = \mathbf{0.75}$$